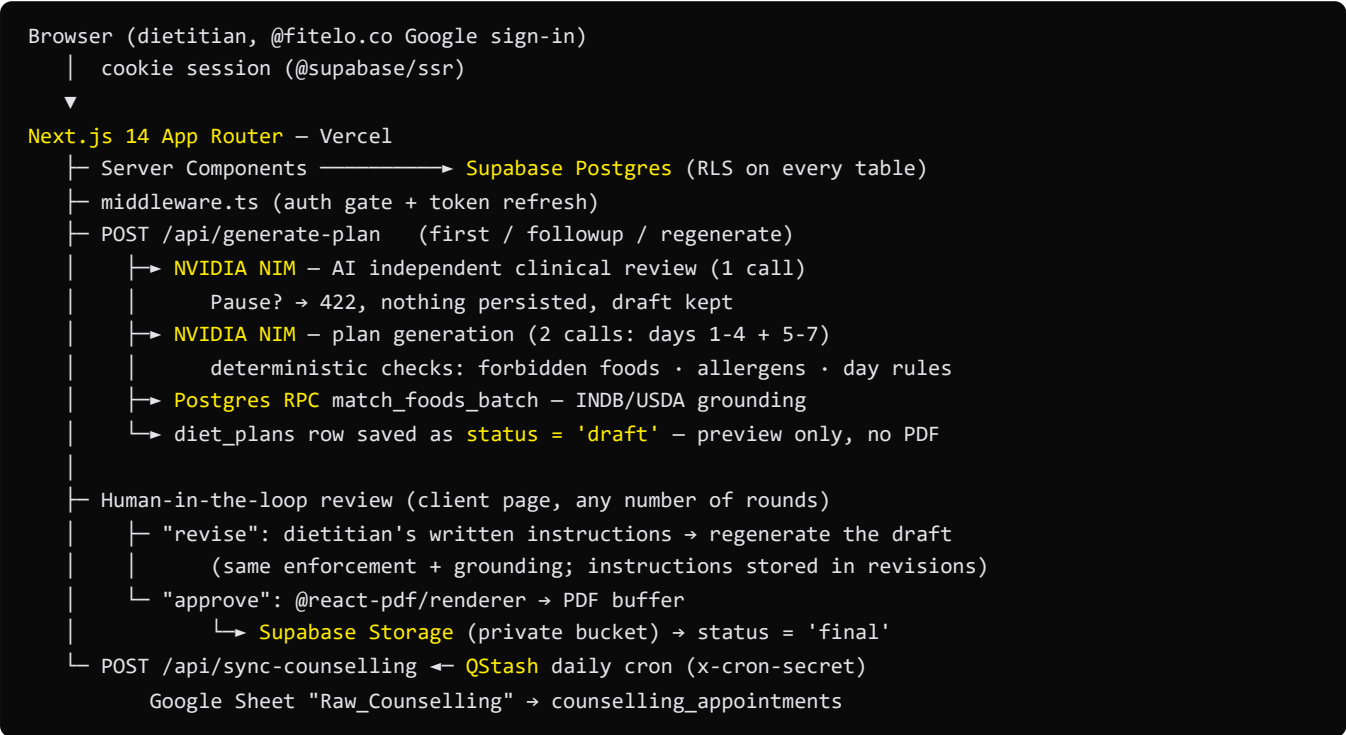


LEANR Diet Platform

Technical Documentation · v2.1 · 16 July 2026



1. ARCHITECTURE OVERVIEW



App reads/writes execute *as the signed-in user* via the cookie-bound Supabase client, so Row Level Security is the source of authorization. The service-role key exists only server-side for the sheet sync and the one-off food seeding script.

Stack

LAYER	TECHNOLOGY
Frontend / SSR	Next.js 14 (App Router), TypeScript, Tailwind CSS (dark LEANR theme, brand yellow #FFED00)
Auth / DB / Files	Supabase — Postgres (+ pg_trgm) + Auth (Google OAuth, fitelo.co-only) + Storage, @supabase/ssr cookie sessions
AI	NVIDIA NIM OpenAI-compatible endpoint — default meta/llama-3.1-70b-instruct , configurable + automatic fallback
Food data	INDB (Indian Nutrient Databank, ICMR-NIN IFCT-based, 1,014 recipes) + USDA SR Legacy (6,781 foods, CC0) seeded into public.foods
Scheduling	Upstash QStash daily schedule → sheet sync endpoint

PDF	@react-pdf/renderer server-side (renderToBuffer)
Hosting	Vercel (Node runtime; generate route maxDuration = 300)

2. REPOSITORY STRUCTURE

```
supabase/migrations/
  0001_init.sql          tables + RLS + signup trigger + storage bucket/policies
  0002_foods.sql         foods reference table + fuzzy-match function
  0003_match_ranking.sql blended match scoring (exact / INDB preference / raw rules)
  0004_google_auth_admin.sql fitelo.co-only signups, profile fields, admin role/policies
  0005_counselling_activity.sql counselling_appointments (ops sheet sync)
  0006_leanr_premium_ai_review.sql diet_plans.ai_review (AI clinical review storage)
  0007_plan_review.sql   diet_plans.status/starts_on/revisions (draft review loop)
scripts/
  seed-foods.mjs         seeds public.foods from scripts/data/*.json
  e2e-test-plan.ts       counselling → review → plan → grounding → PDF, headless
  test-clinical-counselling.mts QA: flags, audit, AI profile, review decision, adherence checks
  test-clients.ts        PRIYA / RAHUL fixtures (answer every mandatory question)
  seed-ui-test-draft.ts  plants a filled counselling as the signed-in dietitian's draft
  test-revision-loop.ts  draft → dietitian instructions → revised draft, with checks
src/
  middleware.ts          session refresh + redirect unauthenticated → /login
  app/
    page.tsx            dashboard (client cards + Today's Activity)
    today/page.tsx      scheduled counselling calls per dietitian emp code
    counselling/new/page.tsx LeanR Premium counselling (loads draft / appointment seed)
    clients/[id]/page.tsx draft review panel, latest plan, AI review card, history, follow-up
    admin/page.tsx      read-only cross-dietitian view + manual sheet sync
    onboarding/page.tsx employee code + phone capture after first Google login
    api/generate-plan/route.ts review + generation pipeline (see §6)
    api/sync-counselling/route.ts sheet → counselling_appointments upsert (QStash)
  lib/
    counselling/questions.ts LeanR Premium bank Q1-Q105 as data (see §4)
    counselling/assessment.ts red flags · 100-pt audit · toIntake() · aiProfile()
    nim.ts                NIM client, review + plan prompts, Zod schemas, enforcement
    nutrition.ts          plan grounding against public.foods (one RPC per plan)
    pdf.tsx              premium macro-report PDF document
    supabase/{client,server}.ts browser / server Supabase factories
    types.ts             IntakeForm, FollowUpInput, row types
  components/           ClinicalCounsellingForm (schema renderer + autosave),
                        PlanReviewControls (draft revise/approve), FollowUpForm,
                        PlanView, ClientCard, SyncSheetButton, ...
```

3. DATA MODEL

TABLE	PURPOSE	KEY COLUMNS
profiles	One row per dietitian; auto-created by trigger; fitelo.co domain enforced	id (= auth.users.id), full_name, employee_code, phone, role (dietitian admin)

<code>clients</code>	Clients owned by a dietitian; the full counselling rides in <code>intake.answers</code>	<code>dietitian_id</code> , basics columns, <code>intake JSONB</code>
<code>diet_plans</code>	One row per weekly plan; created as a draft preview , becomes <code>final</code> (with PDF) only on dietitian approval	<code>client_id</code> , <code>week_number</code> , <code>source</code> , <code>status</code> (draft final), <code>plan JSONB</code> , <code>ai_review JSONB</code> , <code>starts_on</code> (Day 1, fixed at generation), <code>revisions JSONB</code> (dietitian instructions applied, oldest first), <code>pdf_path</code>
<code>followups</code>	Weekly check-in used to generate the next plan	<code>client_id</code> , <code>week_number</code> , <code>weight_kg</code> , <code>adherence</code> , <code>data JSONB</code>
<code>form_drafts</code>	Live autosave of the counselling (answers + source appointment)	<code>unique (dietitian_id, kind)</code> , <code>data JSONB</code>
<code>foods</code>	Nutrition reference — INDB + USDA rows, per-100 g nutrients + household serving weights	<code>name</code> , <code>source</code> , <code>kcal/protein/carbs/fat per 100 g</code> , <code>serving_weights JSONB</code>
<code>counselling_appointments</code>	Ops sheet mirror + workflow status	<code>client_code</code> , <code>emp_code</code> , <code>scheduled_on</code> , sheet columns, <code>status</code> , <code>client_id</code>

RLS: per-dietitian tables use `dietitian_id = auth.uid()`; appointments match `profiles.employee_code = emp_code` (column-grant limits dietitian updates to workflow columns); admin read via `is_admin()`. **Storage:** private bucket `diet-pdfs`, paths `<dietitian_id>/<client_id>/week-N-<ts>.pdf`, policies require the first folder segment to equal `auth.uid()`; downloads via 1-hour signed URLs.

4. COUNSELLING ENGINE (`SRC/LIB/COUNSELLING/`)

The first counselling is the **LeanR Premium Final New Client First Counselling** — Q1–Q105 in 12 sections (goal & motivation · body history · medical safety · digestion · actual food day · preferences & feasibility · training/protein/recovery · routine & behaviour · lifestyle · success/dropout/coaching · dietitian assessment · client discussion), 55–65 minutes, conversational option-select. It is entirely **schema-as-data** in `questions.ts`; `ClinicalCounsellingForm.tsx` renders whatever the schema says, so rewording or adding a question never touches UI code.

- **Question types:** text, textarea, number, time, date, single, multi (with optional `max` — "select up to N", click order doubles as rank), 1–10 sliders.
- **Conditional visibility:** `showIf(answers)` per question — numeric targets appear per selected Q5 target, safety follow-ups on Q21 symptoms, day-of-week rules under Q38, the hormonal section only where relevant, training detail only when training.
- **Meal timeline (Q28):** selecting a meal occasion generates its `q28_<occasion>_{time,food,prep,source,extras}` fields — a structured 24-hour recall.
- **Mandatory gate:** `missingRequired()` (visibility-aware) blocks generation in the UI and again in the API.
- **assessment.ts :** `redFlags()` — urgent Q21 symptoms, severe allergy, disordered-eating risk (Q60), pregnancy, renal/bariatric, RED-S under-fuelling, each marked escalate vs caution; `audit()` — 100-point quality rubric across 11 categories, computed live; `toIntake()` — legacy projection keeping the clients table

and follow-ups unchanged; `aiProfile()` — the 14-block clinical profile the AI analyses (including `dietitian_hypothesis` = Section 11 and `red_flags`).

5. AI PIPELINE (`SRC/LIB/NIM.TS`)

5.1 Independent clinical review (before every Week-1 plan)

Per the LeanR Premium spec, the dietitian's Section 11 strategy is a *professional hypothesis*. `aiClinicalReview()` sends the complete profile and receives a Zod-validated decision object:

```
{ "decision": "Support Dietitian Strategy" | "... With Minor Modification" |
  "Significantly Modify Dietitian Strategy" |
  "Use AI-Led Alternative Clinical Nutrition Strategy" |
  "Pause Final Diet Generation and Request Clinical Review or Missing Information",
  "reasoning": string, "strategy_adjustments": string[],
  "missing_information": string[], "safety_concerns": string[] }
```

A **Pause** aborts generation (HTTP 422) *before* the client row is created, so the counselling remains an editable draft. Any other decision is woven into the plan prompts and stored in `diet_plans.ai_review`. A failed review call degrades to generation-without-review (never blocks the dietitian).

5.2 Plan generation

The shared NIM endpoints cap completions around 2k tokens, so the 7-day plan is generated in **two calls** (overview + days 1–4, then days 5–7 seeing days 1–4 for variety) and merged. Each call is hardened:

`response_format: json_object` with graceful degradation, 120 s per-call timeout, automatic fallback model on transient failures, Zod validation with corrective retries. Truncation is handled explicitly — unparseable output *and* "valid JSON but too few days" (constrained decoding closing the object at the completion cap) both trigger a brevity-focused retry instead of echoing the broken response; if the primary model still cannot produce a valid plan after 3 attempts, the whole conversation is **rerun once on the fallback model** before failing.

Revision rounds: when the dietitian requests changes on a draft (\$7), `generateDietPlan()` receives the current draft plus the written instructions. The prompt marks the instructions as mandatory (second only to allergies and medical safety) and directs the model to revise the existing draft — keeping everything not mentioned — rather than start over. `starts_on` is passed through so weekday-specific rules keep their original Day-1 mapping across revision rounds.

The system prompt encodes the **Final Diet Generation Principles**: never plan from the goal / dietitian strategy / calorie maths / weight alone; obey red flags and doctor instructions; follow the review decision; smallest number of high-impact changes matched to readiness; start from the actual food day; protect favourites, non-negotiables and cultural foods; avoid unnecessary restriction; avoid the client's dropout pattern; fuel muscle, training and recovery; design for the client's actual life.

5.3 Deterministic enforcement (not left to the model)

- **Forbidden foods:** allergens + intolerances + dislikes are listed in the prompt *and* scanned in the returned meals; violations are named back for correction; surviving dislikes are stripped; a surviving **allergen aborts the plan**.
- **Day-of-week rules (q38a–c):** "Day N" maps to real weekdays via `starts_on` (Day 1 = tomorrow, fixed at generation so revision rounds and the approval-time PDF agree); restricted weekdays are checked term-by-term (non-veg, eggs, onion/garlic, all animal products) and violations stripped as a last resort.

- **Every revision round re-runs all of the above** — a dietitian instruction can never reintroduce an allergen, diet-type violation or day-rule breach, and the revised plan is re-grounded against the foods table.

5.4 Nutrition grounding (`src/lib/nutrition.ts`)

All item names go to Postgres in **one RPC** (`match_foods_batch` : `pg_trgm` fuzzy matching, blended scoring with a preference for INDB), quantities like "2 rotis" / "1 katori" / "150 g" resolve to grams via household serving weights, and a meal's macros are replaced *only when every item matched and resolved*. Non-fatal by design: an empty foods table just keeps model estimates.

6. API REFERENCE

POST `/api/generate-plan` (auth: session cookie · Node · maxDuration 300)

BODY	BEHAVIOUR
<code>{"type": "first", "form": ClinicalIntake, "appointmentId": "?"}</code>	Server-side mandatory gate → AI clinical review (Pause → 422 <code>{error, paused, aiReview}</code> , nothing persisted) → create client → generate → ground → insert plan as draft (+ <code>ai_review</code> , <code>starts_on</code>) → mark appointment Completed → clear draft. Returns <code>{clientId, planId, week, status: "draft"}</code> . No PDF yet.
<code>{"type": "followup", "clientId", "followup"}</code>	Inserts the follow-up, updates client weight, generates week <code>latest+1</code> as a draft using the LeanR weekly loop and last week's menu.
<code>{"type": "regenerate", "clientId"}</code>	Retries from stored intake as a draft ; re-runs the AI review when retrying a Week-1 plan.
<code>{"type": "revise", "planId", "instructions"}</code>	Human-in-the-loop round on a draft: rebuilds the original context (intake, AI review, previous week's plan, follow-up data), regenerates with the dietitian's instructions marked mandatory, re-runs enforcement + grounding, updates the same row and appends <code>{instructions, at}</code> to <code>revisions</code> . Draft only — an approved plan returns 409 .
<code>{"type": "approve", "planId"}</code>	Renders the PDF from the reviewed plan (dates from <code>starts_on</code>), uploads to storage, sets <code>status = 'final'</code> + <code>pdf_path</code> . No model calls (~2 s). Draft only — 409 otherwise.

Errors: 400 invalid body/gate · 401 unauthenticated · 404 unknown client/plan · 409 plan already approved · 422 review pause or unparseable stored draft · 502 `{error, clientId?}` on generation/PDF/storage failure (the UI routes to the client page where retry is offered).

Long-request behaviour: generation is a synchronous request (1–3 min typically; longer when the primary model is congested and per-call timeouts cascade). If the browser connection drops mid-request, the server still finishes and saves the draft — refresh the client page instead of resubmitting the counselling form, which would create a duplicate client.

POST /api/sync-counselling

Guarded by `CRON_SECRET` (`x-cron-secret` header or `?secret=`). Reads the Raw_Counselling sheet (Google service account preferred, CSV-export fallback), upserts `counselling_appointments` on (`client_code`, `emp_code`, `scheduled_on`) touching only sheet columns — workflow status survives re-syncs. Triggered daily by QStash and manually from `/admin` .

7. HUMAN-IN-THE-LOOP PLAN REVIEW (`PLANREVIEWCONTROLS`)

Every generated plan — first counselling, follow-up or regenerate — is stored as a **draft preview** and shown on the client page under an amber "Draft preview — needs your review" panel. The dietitian inspects the full plan (meals, macros, repetition, timing) before anything reaches the client. The panel offers:

- **Free-text change instructions** — e.g. *"too much paneer, vary the protein sources; keep protein near 90 g; Day 3 dinner lighter"* — sent with `type:"revise"` . The AI revises the existing draft (not a fresh plan), the same safety enforcement and grounding re-run, and the round is appended to the plan's `revisions` history, displayed as "Changes already applied". Unlimited rounds.
- **Approve & create PDF** — `type:"approve"` renders and stores the final PDF and flips the row to `final` ; the download button appears in plan history.

Rationale: generated diets occasionally repeat foods across the week or overshoot protein; the review checkpoint puts the dietitian's judgement between the model and the client, with their corrections applied by the AI in ~1–2 minutes instead of manual re-editing. Plan history marks unapproved rows "Draft — in review"; drafts have no PDF and never reach a client. The counselling form's autosaved draft is still cleared at generation time (the counselling itself succeeded); plan-level iteration happens entirely on the client page.

8. PDF RENDERING (`SRC/LIB/PDF.TSX`)

- Rendered **at approval time** from the reviewed plan; day dates come from the stored `starts_on` , so what the dietitian approved is exactly what the PDF shows.
- Premium macro-report layout: black header band with client name + LEANR logo, macro target cards, per-day tables (Time | Meal | Foods | Calories | Protein | Carbs | Fat | Cal%), highlighted Daily Total rows, Weekly Summary with P%/C%/F%, guidelines, foods to avoid, real dates per day, and a two-line disclaimer footer on every page.
- Daily totals, Cal% and the weekly summary are *computed from meal data*, never trusted from the model.
- Preview without the AI: `npx tsx scripts/render-sample-pdf.tsx out.pdf` .

9. AUTOSAVE DESIGN

`ClinicalCounsellingForm` debounces every change (1.2 s) into an upsert on `form_drafts` keyed by (`dietitian_id`, `kind`) , storing answers + the source appointment id, with a live Saving/Saved/Failed indicator. The draft restores server-side on load (only for the same appointment) and is deleted exclusively after successful generation — a failed generation, review Pause, or closed tab never loses call notes.

10. ENVIRONMENT VARIABLES

VARIABLE	SCOPE	NOTES
NEXT_PUBLIC_SUPABASE_URL / NEXT_PUBLIC_SUPABASE_ANON_KEY	client + server	RLS makes the anon key safe to expose
SUPABASE_SERVICE_ROLE_KEY	server only	Sheet sync + <code>npm run seed:foods</code> ; never in the browser
NVIDIA_API_KEY	server only	From build.nvidia.com
NVIDIA_MODEL / NVIDIA_FALLBACK_MODEL / NVIDIA_NIM_URL	server, optional	Defaults: llama-3.1-70b / llama-3.1-8b / integrate.api.nvidia.com. Current config: <code>mistral-small-4</code> primary (fast; occasional malformed JSON is handled by validation retries) with <code>llama-3.1-70b</code> fallback (reliable but congestion-prone). The fallback covers transient errors <i>and</i> persistent validation failures.
GOOGLE_SERVICE_ACCOUNT_EMAIL / GOOGLE_SERVICE_ACCOUNT_KEY	server only	Sheets API read access to Raw_Counselling (share sheet as Viewer)
COUNSELLING_SHEET_ID / COUNSELLING_SHEET_RANGE / COUNSELLING_SHEET_CSV_URL	server only	Range defaults to <code>Raw_Counselling</code> ; CSV URL is the link- viewable fallback
CRON_SECRET	server only	Shared secret QStash sends to the sync endpoint

11. SETUP & DEPLOYMENT

- Supabase:** run migrations `0001` → `0007` in order in the SQL Editor (tables, RLS, trigger, bucket/policies, foods matching, Google-auth constraints, appointments, `ai_review`, draft-review columns).
- Food data:** put the service-role key in `.env.local`, run `npm run seed:foods` (~7,900 rows; idempotent upsert).
- Google sign-in:** create an OAuth client in Google Cloud Console, add Supabase's redirect URI, enable the Google provider in Supabase Auth.
- Local:** `npm install` → fill `.env.local` from `.env.example` → `npm run dev`.
- Vercel:** import the repo, set env vars, deploy; add the domain to Supabase Auth URL configuration; create the QStash daily schedule (POST `/api/sync-counselling`, cron `30 0 * * *`, secret header).
- QA:** `npx tsx scripts/test-clinical-counselling.mts` (flags → audit → review → plan → adherence checks), `npx tsx scripts/e2e-test-plan.ts` (two clients → PDFs in `test-output/`) and `npx tsx scripts/test-revision-loop.ts` (draft → written instructions → revised draft, with protein/variety checks).

12. OPERATIONS & TROUBLESHOOTING

SYMPTOM	CAUSE / FIX
422 "AI clinical review paused diet generation"	Working as designed — the review found missing information or a safety concern. The counselling is still a draft: address the listed gaps (or the flagged risk) and resubmit.

Generation slow or 502 "NVIDIA NIM ... failed (504)"	Shared free NIM endpoint congested. The fallback model usually succeeds; optionally point <code>NVIDIA_MODEL</code> at a healthier model. Retry from the client page — intake is never lost.
Browser shows a network error mid-generation	The server finishes independently — refresh the client page after a minute; the draft usually appears. Do <i>not</i> resubmit the counselling form (creates a duplicate client).
"AI (model) returned an invalid diet plan after 3 attempts"	Model returned malformed/truncated JSON repeatedly; the run already retried with brevity demands and once on the fallback model. Usually transient — regenerate; if persistent, switch <code>NVIDIA_MODEL</code> .
Plan stuck as "Draft — in review"	Working as designed — the dietitian hasn't approved it. Open the client page, revise or approve from the amber panel.
Insert fails with unknown column <code>status</code> / <code>starts_on</code> / <code>revisions</code>	Migration <code>0007_plan_review.sql</code> not yet applied.
"AI could not produce an allergen-safe plan"	The allergen kept reappearing across corrective retries, so the plan was refused. Regenerate; persistent cases usually mean an over-broad allergen term.
"N mandatory counselling questions unanswered"	Server-side gate — open the sections marked in red; required questions are visibility-aware.
Plan macros look like model estimates	Foods table empty/missing or items didn't match — check <code>npm run seed:foods</code> ran and the grounding log line (<code>nutrition grounding: X/Y meals</code>).
Insert fails with unknown column <code>ai_review</code>	Migration <code>0006_leanr_premium_ai_review.sql</code> not yet applied.
Today's Activity empty	Dietitian's <code>employee_code</code> doesn't match the sheet's Emp Code, or the sync hasn't run — trigger it from <code>/admin</code> .
PDF download does nothing	Pop-up blocked, or storage policies missing — re-run the <code>storage.objects</code> policy section of 0001.

Security summary: app traffic rides the user's JWT through RLS; the service-role key is confined to the sync endpoint and local seeding; the private bucket is only reachable through per-user policies and short-lived signed URLs; signups are domain-locked to fitelo.co; the NVIDIA key, sheet credentials and cron secret live server-side only.